1 Q1: Structural Encoding in Graph Transformer

`LapPETransformer` adds Laplacian eigenvector positions to every node once, before the first block; `SPDAttention` adds a learnable per-head scalar, indexed by the pair's hop distance, to every attention score. Two choices in the absolute encoding matter.

Which eigenvectors, and at what scale. The smallest eigenvalue of $L = I - D^{-1/2}AD^{-1/2}$ is 0 with the constant eigenvector $D^{1/2}\mathbf{1}$, the same for every connected graph, so `precompute` drops it and returns columns $1, \ldots,$ `LAP_K`; a graph with fewer is zero-padded (ZINC's smallest molecule has 9 nodes, so exactly 8 exist). Each column is then scaled by $\sqrt{n}$: eigenvector entries are $O(n^{-1/2})$, so unscaled a channel has RMS 0.011 on the 7624-node LastFM graph, where `num_atom_types=0` leaves the encoding as the model's only input.

Sign ambiguity. v and $-v$ are the same eigenvector and the solver picks one arbitrarily, so each graph's channels get an independent random sign flip while training and the identity at evaluation. That is an augmentation *across* graphs, so it is skipped when the batch holds only one — on LastFM it would be noise on the model's only input.

The relative encoding computes all-pairs hop distances by frontier expansion: exact on ZINC, blocked and stopped after `max_dist` hops for LastFM's 7624×7624 matrix, which costs nothing because every distance from `max_dist` upwards shares one bias; unreachable pairs keep their own row. The lookup is added to the scores before mask and softmax, $\text{score}_{ij} = q_i^\top k_j/\sqrt{d} + b_{\text{SPD}(i,j)}$, which is Graphormer's form. The distances were checked against a plain BFS, the eigenvectors against the dense spectrum of L.

Table 1: ZINC subset, 30 epochs, dense attention; 4 blocks, dim 64, 4 heads, the framework's fixed seed. Epoch selected on validation MAE.

encoding	val MAE	test MAE	best epoch
none (plain `Attention`)	0.6472	0.7013	18
demo: degree centrality + common neighbours	0.4629	0.4957	27
LapPE only (absolute)	0.6205	0.6575	30
SPD only (relative)	0.5373	0.5614	29
LapPE + SPD (cell 14)	**0.5160**	**0.5496**	30

Both encodings help and SPD carries almost all of it: plain attention is at 0.7013 test MAE, SPD alone at 0.5614, LapPE alone at 0.6575. Putting LapPE on top of SPD moves it by 0.0119, inside the ± 0.0100 seed-to-seed half-range Table 2 measures for that same model, so this budget does not establish a LapPE effect beyond SPD. Both choices above were ablated on both datasets, and they behave differently. The $\sqrt{n}$ scaling is invisible on ZINC (0.5449 against 0.5496) and worth 0.0508 micro-F1 on LastFM. The sign flip is worth 0.0503 test MAE on ZINC and costs 0.1281 micro-F1 on LastFM, which is why it is conditional.

Implementation (notebook cell 13, with its two helpers)

```
SPD_MAX_DIST = 8          # distances above this share one bias (SPDAttention.max_dist)
DENSE_LIMIT = 1024        # graphs this small are handled with dense linear algebra


def laplacian_eigvecs(edge_index, num_nodes, k=LAP_K):
    """Rows of the k eigenvectors of L = I - D^-1/2 A D^-1/2 with the smallest
    non-trivial eigenvalues, scaled by sqrt(n) so each channel has unit RMS.
```

```
    Padded with zero columns when the graph has fewer than k non-trivial eigenvectors."""
    n = num_nodes
    A = sp.coo_matrix((np.ones(edge_index.size(1)), (edge_index[0].numpy(), edge_index[1].numpy())),
                      shape=(n, n)).tocsr()
    dinv = 1.0 / np.sqrt(np.maximum(np.asarray(A.sum(1)).ravel(), 1.0))
    Ahat = sp.diags(dinv) @ A @ sp.diags(dinv)          # = I - L, same eigenvectors
    m = min(k, n - 1)                                   # non-trivial eigenvectors available
    out = np.zeros((n, k), dtype=np.float32)
    if m > 0:
        if n <= DENSE_LIMIT:
            w, V = np.linalg.eigh(np.eye(n) - Ahat.toarray())           # ascending
        else:                                            # smallest of L = largest of I - L
            v0 = np.random.default_rng(RANDOM_SEED).standard_normal(n)   # fixed ARPACK start
            w, V = sp.linalg.eigsh(Ahat, k=m + 1, which="LA", tol=0, v0=v0)
            w, V = 1.0 - w, V
            order = np.argsort(w)
            w, V = w[order], V[:, order]
        out[:, :m] = V[:, 1:m + 1] * math.sqrt(n)        # skip the trivial eigenvector
    return torch.from_numpy(out)


def hop_distances(edge_index, num_nodes, cap=None):
    """All-pairs hop distance by frontier expansion, as int16 [n, n].

    Unreachable pairs (different connected components) are -1.  With `cap` set the
    expansion stops after `cap` hops and every pair that is further away is reported
    as `cap`; those pairs share the max_dist bias anyway."""
    n = num_nodes
    src, dst = edge_index[0].numpy(), edge_index[1].numpy()
    A = sp.coo_matrix((np.ones(len(src), dtype=np.float32), (src, dst)), shape=(n, n)).tocsr()
    D = np.full((n, n), -1, dtype=np.int16)
    limit = n if cap is None else cap
    if n <= DENSE_LIMIT:
        Ad = np.zeros((n, n), dtype=np.float32)
        Ad[src, dst] = 1.0
        seen = np.eye(n, dtype=bool)
        frontier = seen.copy()
        D[seen] = 0
        for d in range(1, limit + 1):
            frontier = ((frontier.astype(np.float32) @ Ad) > 0) & ~seen
            if not frontier.any():
                break
            D[frontier] = d
            seen |= frontier
    else:
        for s0 in range(0, n, 1024):                     # blocks of sources, columns of F
            s1 = min(s0 + 1024, n)
            F = np.zeros((n, s1 - s0), dtype=np.float32)
            F[np.arange(s0, s1), np.arange(s1 - s0)] = 1.0
            seen = F.astype(bool)
            D[np.arange(s0, s1), np.arange(s0, s1)] = 0
            for d in range(1, limit + 1):
                F = ((A @ F) > 0) & ~seen
                if not F.any():
                    break
                D[s0:s1][F.T] = d
                seen |= F
                F = F.astype(np.float32)
    if cap is not None:                                  # reached later than the cap, not unreachable
        comp = sp.csgraph.connected_components(A, directed=False)[1]
        far = (D < 0) & (comp[:, None] == comp[None, :])
        D[far] = cap
    return torch.from_numpy(D)

class LapPETransformer(GraphTransformer):
    """Absolute encoding: Laplacian eigenvectors, added to the input once."""

    def __init__(self, *args, **kwargs):
```

```
        super().__init__(*args, **kwargs)
        # Linear map from the LAP_K eigenvector entries of a node to a node vector of size dim.
        self.lap_proj = nn.Linear(LAP_K, self.dim)

    @staticmethod
    def precompute(edge_index, num_nodes):
        return {"lap": laplacian_eigvecs(edge_index, num_nodes)}

    def inject_absolute(self, h, batch):
        lap = batch["lap"]
        # Eigenvectors are defined up to sign, so flip each graph's channels at random while
        # training and use the identity at evaluation.  This is an augmentation ACROSS graphs:
        # on a single-graph batch there is nothing to generalise over and the flip is pure
        # input noise, so it is skipped (it costs ~0.12 micro-F1 on LastFM).
        if self.training and batch["num_graphs"] > 1:
            sign = torch.randint(0, 2, (batch["num_graphs"], lap.size(1)),
                                 device=lap.device, dtype=lap.dtype) * 2 - 1
            lap = lap * sign[batch["batch"]]
        return h + self.lap_proj(lap)


class SPDAttention(Attention):
    """Relative encoding: shortest-path distance, one learnable bias per head."""

    def __init__(self, dim, heads, max_dist=SPD_MAX_DIST):
        super().__init__(dim, heads)
        self.max_dist = max_dist
        # one bias per head for each distance 0..max_dist, plus one for unreachable pairs
        self.spd_table = nn.Embedding(max_dist + 2, heads)

    @staticmethod
    def precompute(edge_index, num_nodes):
        cap = None if num_nodes <= DENSE_LIMIT else SPD_MAX_DIST
        return {"spd": hop_distances(edge_index, num_nodes, cap=cap)}

    def bucket(self, d):
        """Hop distance -> row of spd_table: 0..max_dist, and max_dist + 1 if unreachable."""
        d = d.long()
        return torch.where(d < 0, torch.full_like(d, self.max_dist + 1), d.clamp(max=self.max_dist))

    def forward(self, h, enc, mask):
        B, N, D = h.shape
        q, k, v = self.qkv(h).view(B, N, 3, self.heads, D // self.heads).permute(2, 0, 3, 1, 4)
        scores = q @ k.transpose(-1, -2) / math.sqrt(D // self.heads)      # [B, H, N, N]
        bias = self.spd_table(self.bucket(enc["spd"]))                     # [B, N, N, H]
        scores = scores + bias.permute(0, 3, 1, 2)
        scores = scores.masked_fill(~mask[:, None, None, :], float("-inf"))
        weights = torch.softmax(scores, dim=-1).nan_to_num(0.0)
        return self.out((weights @ v).transpose(1, 2).reshape(B, N, D))
```

2 Q2: Efficient Graph Transformer

`SparseSPDAttention` keeps Q1's parameters and `precompute` and changes only which pairs exist. Five pair sets were built and compared (Table 2): the graph's edges alone; plus one global virtual node; plus Exphormer's random expander at two widths; and the 2-hop ball — every ordered pair at most two hops apart — plus the virtual node. The submitted class attends to the last of these, picked on validation. Its cost is $\sum_v \deg(v)^2$ rather than n^2: 189 pairs per ZINC molecule, 804,349 on LastFM (1.4% of full). The expander was the a-priori favourite — it alone keeps the pair count $O(n)$ while mixing in $O(\log n)$ hops — but both widths finished last on both datasets.

`forward` gathers q, k, v at the endpoints of each pair, adds the inherited SPD bias plus one learnable bias per pair type, normalises with `segment_softmax` over the pairs that share a querying

node, and scatters the weighted values back with `index_add_`. The handout points at a `segment_attention` helper that does not exist in PyG 2.8.1; cell 3 imports `softmax as segment_softmax`, and that is what is used. The framework also reports `enc["spd"]` = 0 for pairs touching a virtual node, which would alias them with genuine distance-0 self loops, so those go to the unreachable row of `spd_table` instead. On a complete pair set this path reproduces the dense Q1 attention to 7.5×10^{-8}.

Table 2: Masking strategies on ZINC, same model and budget as Table 1, with LapPE throughout. Pairs are per graph, averaged over the 10,000 training molecules ($n^2 = 557$ on average). The val and test columns are the framework's seed, as cell 18 runs it; the last column is the mean and half-range of the test MAE over that seed and two more, the expander rows redrawing their expander each time. Re-running the submitted row three times with every input identical — same seed, same weights, same batch order — gives test MAE 0.4766, 0.4766, 0.4766: the sparse forward accumulates with `index_add_`, whose order across CPU threads is not fixed, so one ZINC number here is good to about ±0.01. All of this compares pair sets inside this framework — 30 epochs, and the bond features `make_batch` carries are never read — rather than reproducing any paper's numbers.

attention pairs	pairs/graph	of full	val MAE	test MAE	3-seed mean
self loops + graph edges	73	13%	0.4918	0.5048	0.5145 ± 0.0133
+ virtual node	120	22%	0.4791	0.5047	0.5092 ± 0.0067
2-hop ball + virtual (submitted)	189	34%	**0.4293**	**0.4677**	0.4909 ± 0.0211
+ 4-regular expander	200	36%	0.5179	0.5551	0.5708 ± 0.0129
+ 8-regular expander	266	48%	0.5213	0.5554	0.5655 ± 0.0119
full attention (Q1)	557	100%	0.5160	0.5496	0.5614 ± 0.0100

ZINC says the ranking is not the one the pair counts suggest. Restricting attention to the graph beats full attention outright — 0.5092 against 0.5614 test MAE on the three-seed mean — so on 23-atom molecules the sparsity is a useful prior, not a sacrifice. Two hops wins again, at 3 of the 3 seeds and by 0.0184 on the mean, while a random expander of comparable size loses 0.0799: the pairs have to mean something, and an expander edge means nothing to a molecule. LastFM neither contradicts nor confirms that — its top three finish within 0.008 validation micro-F1, the submitted set second of five — so ZINC decides. What LastFM shows instead is the cost: 0.33 s a step against 5.24 s for full attention over all 58,125,376 pairs, a 16× saving, at test micro-F1 0.7827.

Table 3: LastFM (7624 nodes, 27,806 edges, 18 countries), 2 blocks, dim 32, 2 heads, CPU, 8 threads. One training step is the best of 7 forward-backward passes; micro-F1 is `run_node_task` over 200 epochs, selected on validation. Full attention is not trained for 200 epochs: at 5.24 s a step it would cost more than every other experiment here together. The framework's peak-memory column prints `n/a on CPU`, so no memory figure is quoted.

attention pairs	pairs attended	of full	one step (s)	test micro-F1
full attention (Q1)	58,125,376	100.000%	5.24	—
self loops + graph edges	63,236	0.109%	0.03	0.7893
+ virtual node	78,485	0.135%	0.04	0.7826
2-hop ball + virtual (submitted)	804,349	1.384%	0.33	**0.7827**
+ 4-regular expander	108,965	0.187%	0.05	0.7678
+ 8-regular expander	139,417	0.240%	0.06	0.7607

Implementation (notebook cell 16)

```
EXPANDER_CYCLES = 2       # union of 2 random Hamiltonian cycles = a 4-regular expander


def pair_set(edge_index, num_nodes, n_cycles=EXPANDER_CYCLES, virtual=True, extra=()):
    """The pair set of one graph: self loops, the graph's edges, an expander built as the
    union of n_cycles random Hamiltonian cycles, and one global virtual node (id num_nodes).
    The cycles are drawn from a generator seeded by the graph itself, so they are fixed."""
    n, node = num_nodes, torch.arange(num_nodes)
    gen = torch.Generator().manual_seed((RANDOM_SEED * 1000003 + n * 7919
                                         + int(edge_index.sum())) % (2 ** 31))
    parts = [torch.stack([node, node]), edge_index, *extra]        # self loops + graph edges
    for _ in range(n_cycles if n >= 3 else 0):                     # expander edges
        p = torch.randperm(n, generator=gen)
        q = torch.roll(p, -1)
        parts += [torch.stack([p, q]), torch.stack([q, p])]
    if virtual:                                                    # global node, both directions
        virt = torch.full_like(node, n)
        parts += [torch.stack([node, virt]), torch.stack([virt, node]), torch.tensor([[n], [n]])]
    return coalesce(torch.cat(parts, 1), num_nodes=n + 1)


class SparseSPDAttention(SPDAttention):
    """Your Q1 attention, restricted to a chosen set of node pairs.

    Of the five pair sets measured (see the report) the 2-hop ball plus one global virtual node
    had the lowest validation MAE on ZINC and the highest validation micro-F1 on LastFM, so that
    is what this class attends to.  pair_set below still builds Exphormer's random expander; the
    expander variants are one argument away and are compared in the report."""

    def __init__(self, dim, heads, max_dist=SPD_MAX_DIST):
        super().__init__(dim, heads, max_dist)
        # Exphormer gives each kind of attention edge its own bias: local, expander, global
        self.type_table = nn.Embedding(len(PAIR_TYPES), heads)

    @staticmethod
    def attention_pairs(edge_index, num_nodes):
        """Every ordered pair at most two hops apart, plus one global virtual node."""
        n = num_nodes
        A = sp.coo_matrix((np.ones(edge_index.size(1)), (edge_index[0].numpy(), edge_index[1].numpy())),
                          shape=(n, n)).tocsr()
        ball = ((A + A @ A) > 0).tocoo()                   # pairs at distance 0, 1 or 2
        extra = torch.from_numpy(np.stack([ball.row, ball.col])).long()
        return pair_set(edge_index, num_nodes, n_cycles=0, virtual=True, extra=[extra])

    def forward(self, h, enc, pairs):
        M, D = h.shape
        H, dh = self.heads, D // self.heads
        src, dst = pairs                                   # node dst attends to node src
        q, k, v = self.qkv(h).view(M, 3, H, dh).permute(1, 0, 2, 3)
        score = (q[dst] * k[src]).sum(-1) / math.sqrt(dh)          # [P, H]
        virtual = enc["pair_type"] == PAIR_TYPES.index("virtual")  # their spd arrives as 0,
        d = torch.where(virtual, torch.full_like(enc["spd"], -1), enc["spd"])  # but no distance
        score = score + self.spd_table(self.bucket(d)) + self.type_table(enc["pair_type"])
        alpha = segment_softmax(score, dst, num_nodes=M)           # softmax over each node's pairs
        out = torch.zeros(M, H, dh, device=h.device, dtype=h.dtype)
        out.index_add_(0, dst, alpha.unsqueeze(-1) * v[src])       # [M, H, dh]
        return self.out(out.reshape(M, D))
```